

MBEYA UNIVERSITY OF SCIENCE AND TECHNOLOGY



GROUP ASSIGMENT: GROUP LINUX

PROGRAMME: BACHELOR OF COMPUTER SCIENCE

COURSE: APPLICATION DEVELOPMENT FOR MOBILE DEVICES

COURSE CODE: CS 8306

S/N	NAME	REG NO.
01	JUMA MUSTAFA MSHIHIRI	23100533350030
02	NDEGE KILYABOYA	23100533350088
03	SITRATON MMASI	23100533350027
04	PRECIOUS LOBSTER	23100533350062

PROJECT REPORT

1. Introduction

Problem Description

The hotel industry requires accurate booking and reservation efficient room management. In many small hotels the booking and reservations process are sometimes recorded manually or through simple tools such as spreadsheets. These methods may result in booking conflicts, data inconsistency, and difficulty restoring past records.

This project addresses these issues by developing a web application for Hotel Management System that is responsible for room management and reservation handling in a structured and reliable manner.

Motivation

The motivation for this project was both practical and academic. Practically, hotels require systems that reduce manual errors and improve operational efficiency. Academically, this project allowed the application of key web development principles.

The system would demonstrates how modern web technologies can be combined to build a secure and scalable application.

Project Objectives

Main objective

- To develop a hotel management system

Specific objectives

- I. To develop customers and admin user interface
- II. To develop a database for storing rooms details, reservations, admin profile and passwords
- III. To develop a booking functionality for the customers to book for rooms
- IV. To develop confirming and canceling functionality for the admin

2. REQUIREMENTS(FUNCTIONAL&NON-FUNCTIONAL)

Functional Requirements

- **View Room Details**
The system shall allow customers to view details of a selected room, including room number, type, price per night, capacity, status, and description.
- **Check Room Availability by Dates**
The system shall allow customers to check which rooms are available for a selected check in and check out date range.
- **Create a Reservation**
The system shall allow customers to create a reservation by submitting guest details and booking dates for a selected room.
- **Validate Reservation Input**
The system shall validate reservation input before submission, ensuring required fields are provided and the checkout date is after the checkin date.
- **Store Reservation Records**
The system shall store reservation information in the database and link it to the appropriate room using a foreign key relationship.
- **Display Reservation Submission Result**
The system shall display a success message when a reservation is created successfully and display an error message when reservation creation fails.

Administrator (Admin User) Requirements

- **Admin Login**
The system shall allow an administrator to log in using an email and password.
- **Generate Authentication Token**
The system shall generate a JWT token after successful login and return it to the admin client for authenticated access.
- **View All Reservations**
The system shall allow an authenticated admin to retrieve and view all reservations from the database.
- **Filter Reservations by Status**
The system shall allow an authenticated admin to filter reservations based on status eg pending, confirmed, cancelled.

- **Confirm Reservation**
The system shall allow an authenticated admin to confirm a reservation by updating its status to confirmed.
- **Cancel Reservation**
The system shall allow an authenticated admin to cancel a reservation by updating its status to cancelled.
- **Display Reservation Update Result**
The system shall display a success message when a reservation is confirmed or cancelled successfully and display an error message when the action fails.
- **Admin Logout**
The system shall allow an admin to log out by clearing the stored authentication token.

Non-Functional Requirements

Security Requirements

- **Secure Password Storage**
The system shall store administrator passwords using a secure one-way hashing algorithm (bcrypt) and shall not store plain text passwords.
- **Authentication and Authorization**
The system shall restrict access to admin features using token based authentication (JWT) and deny requests with missing or invalid tokens.
- **Data Protection in API Communication**
The system shall exchange data between frontend and backend using structured JSON and should be deployed with HTTPS in a production environment to protect data in transit.
- **SQL Injection Prevention**
The system shall use parameterized SQL queries to reduce the risk of SQL injection attacks.

Performance Requirements

- **A good Response Time**
The system shall return responses for common operations (room listing, reservation creation, reservation listing) within a reasonable time under normal usage conditions.

- **Efficient Database Access**
The system shall use efficient queries and proper indexing (eg primary keys and foreign keys) to support fast retrieval of rooms and reservations.

Reliability Requirements

- **Data Persistence**
The system shall ensure that room and reservation data are stored persistently in the database and remain available after server restarts.
- **Error Handling and Stability**
The system shall handle runtime errors gracefully and return meaningful error messages without crashing the server.

Maintainability Requirements

- **Modular Code Structure**
The system shall use a modular architecture (separate routes, controllers, middleware, and database configuration) to make maintenance and updates easier.
- **Readable and Consistent Code Style**
The system shall maintain consistent naming conventions and code structure to improve readability and teamwork efficiency.

Scalability Requirements

- **Future Feature Expansion**
The system shall be designed in a way that allows future features such as payment integration, email notifications, or multiple user roles to be added with minimal restructuring.
- **Deployment**
The system should support deployment to a production environment with minimal changes, such as moving configuration to environment variables and using a managed database.

3. SYSTEM DESIGN & ARCHITECTURE

I. Overall System Design

The system follows a three tier architecture:

1. Presentation Layer (Frontend)

It handles user interaction. It is built using HTML, CSS, and JavaScript, and provides separate interfaces for customers and administrators

2. Application Layer (Backend)

It is built with Node.js and Express.js. It contains routes, controllers, middleware, and database configuration files. The backend processes requests, applies business logic, handles authentication, and communicates with the database

3. Data Layer (Database)

It uses MySQL to store rooms, reservations, and admin credentials. The backend connects to the database and performs CRUD operations using SQL queries

This structured separation ensures better organization, maintainability, and scalability of the system.

II. Client-Server Architecture Diagram

Client (Browser)



HTTP Requests

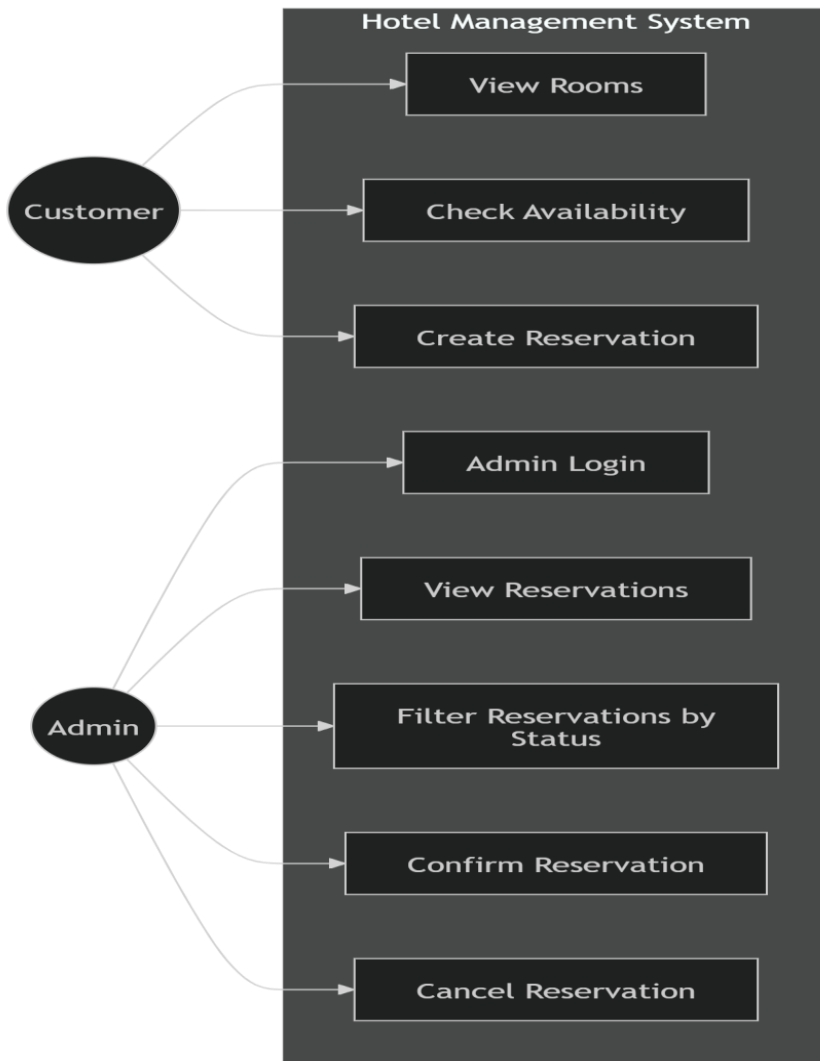
Backend Server (Node.js + Express)



SQL Queries

MySQL Database

A USE CASE DIAGRAM



III. Database Schema

Rooms Table

- id (Primary Key)
- room_number
- type
- price_per_night
- capacity

- status
- description
- created_at
- updated_at

Reservations Table

- id (Primary Key)
- room_id (Foreign Key → Rooms.id)
- guest_name
- guest_phone
- guest_email
- check_in
- check_out
- adults
- children
- status
- created_at

Admins Table

- id (Primary Key)
- email (Unique)
- password_hash
- created_at

IV. API Overview

The system uses a RESTful API built with Node.js and Express.js to handle communication between the frontend and the database. The API allows the frontend to send HTTP requests and receive responses in JSON format.

The API is divided into two main parts:

- Customer APIs:-These allow users to view rooms, check availability, and create reservations.
- Admin APIs:- These allow authenticated administrators to log in, view reservations, confirm bookings, and cancel reservations.

Standard HTTP methods are used:

- GET for retrieving data
- POST for creating reservations
- PUT/PATCH for updating reservation status

Admin endpoints are protected using JWT authentication to ensure only authorized users can access them.

Overall, the API acts as the middle layer that processes requests, applies business logic, and interacts with the MySQL database

4. SECURITY AND AUTHENTICATION

I. Log In Mechanism

The login mechanism is implemented for administrators using secure authentication. When an admin enters their email and password, the frontend sends the credentials to the backend through a login API endpoint. The backend retrieves the stored password hash from the database and compares it with the entered password using bcrypt.

If the credentials are valid, the system generates a JWT token and sends it back to the frontend. The frontend stores this token and includes it in the Authorization header when accessing protected admin routes.

Before allowing access to any admin endpoint, the backend verifies the token using middleware. If the token is missing or invalid, access is denied.

II. How Authentication Is Handled

JWT tokens are issued after login and verified before granting access to protected routes.

III. How Passwords Are Stored

Passwords are firstly hashed using bcrypt then after that they are being stored in the database , hence some one can not access the passwords

IV. Token/Session Handling

The system uses JWT (JSON Web Tokens) for session handling instead of traditional server side sessions.

After a successful admin login, the backend generates a JWT token and sends it to the frontend. The frontend stores this token (for example, in localStorage) and includes it in the backend when making requests to protected admin endpoints.

Each time a protected request is made, the backend verifies the token using middleware. If the token is valid, access is granted if it is missing, invalid, or expired, the request is rejected.

V. How Data Is Protected

Data is being protected through several techniques such as

- Protected admin endpoints
- Environment variables for secrets
- SQL parameterization
- There is middleware verification

VI. Access Control Implementation

Access control is implemented by separating customer and admin functionalities.

- Customer routes are public and allow viewing rooms and creating reservations.
- Admin routes are protected and require a valid JWT token.
- Middleware checks the token before the request reaches the controller.
- If authentication fails, the server returns a 401 Unauthorized response.

This ensures that only authenticated administrators can manage reservations, while customers cannot access restricted endpoints.

5. Implementation Details

I. Technologies Used

In frontend we have used HTML, CSS and JavaScript

In backend we have used Node.js and Express.js

In database we have used MySQL implemented in the XAMPP environment

II. Frontend Implementation

The frontend is divided into customer and admin sections in which the

Customer pages allow room viewing and booking.

Admin pages allow reservation management.

Html has been used in structuring the pages while css used in styling and JavaScript for responsive feature

III. Backend Implementation

The backend uses a structure of

- Routes for API endpoints
- Controllers for business logic
- Middleware for authentication
- Db for database connection

IV. Database Design

The database structure avoids redundancy.

Rooms and reservations are separated to maintain normalization.

Foreign key constraints ensure referential integrity.

V. Challenges Faced

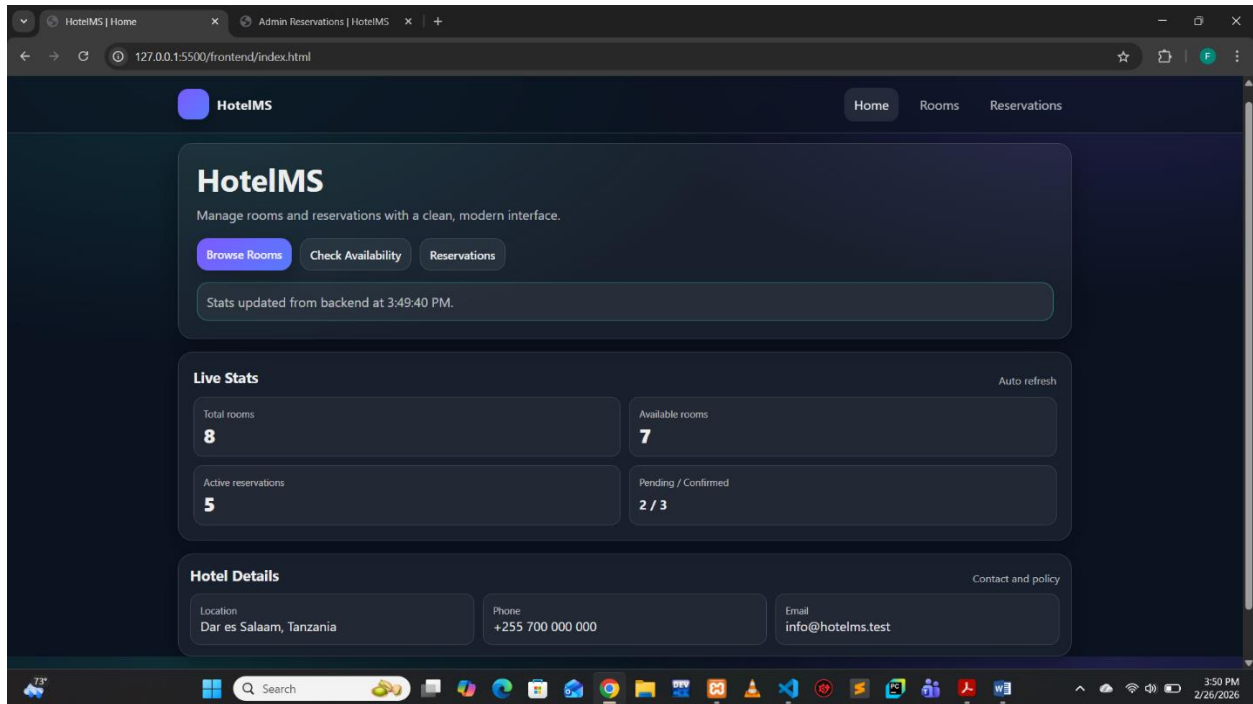
- Correct separation of admin and customer APIs
- Debugging the failure of css loading
- Implementing JWT authentication
- Managing secure password storage

VI. Assumptions Made

- System is designed for a single hotel
- Only one admin role exists
- No payment integration
- Internet access is available

6. RESULTS, SCREENSHOTS & CONCLUSION

I. Screenshots for the application



- Figure 1: Home Page

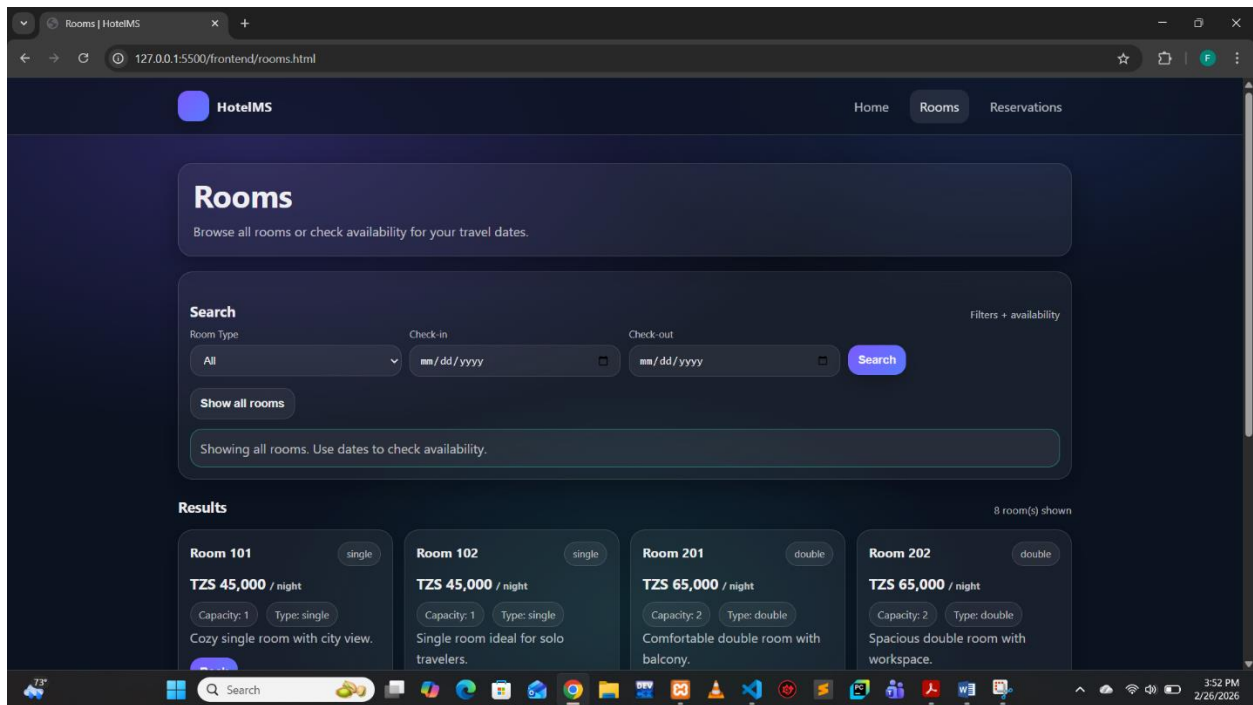


Figure 2: Rooms Page

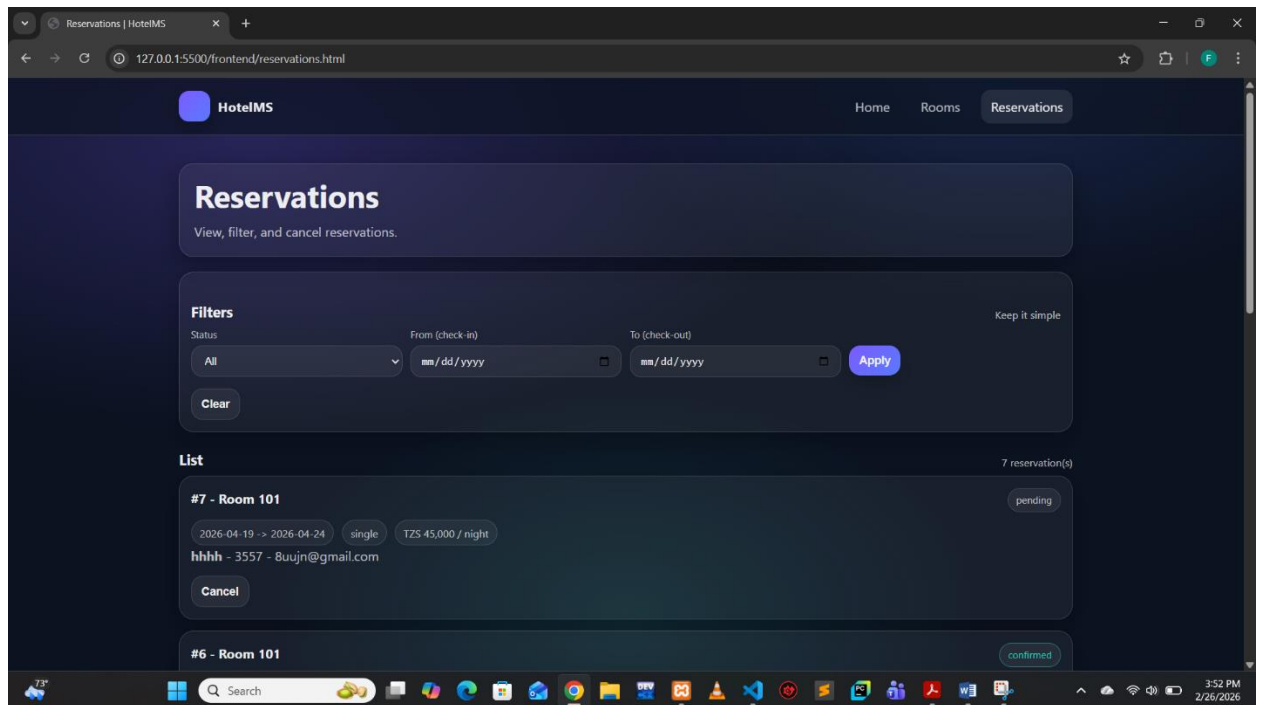


Figure 3: Reservation Form

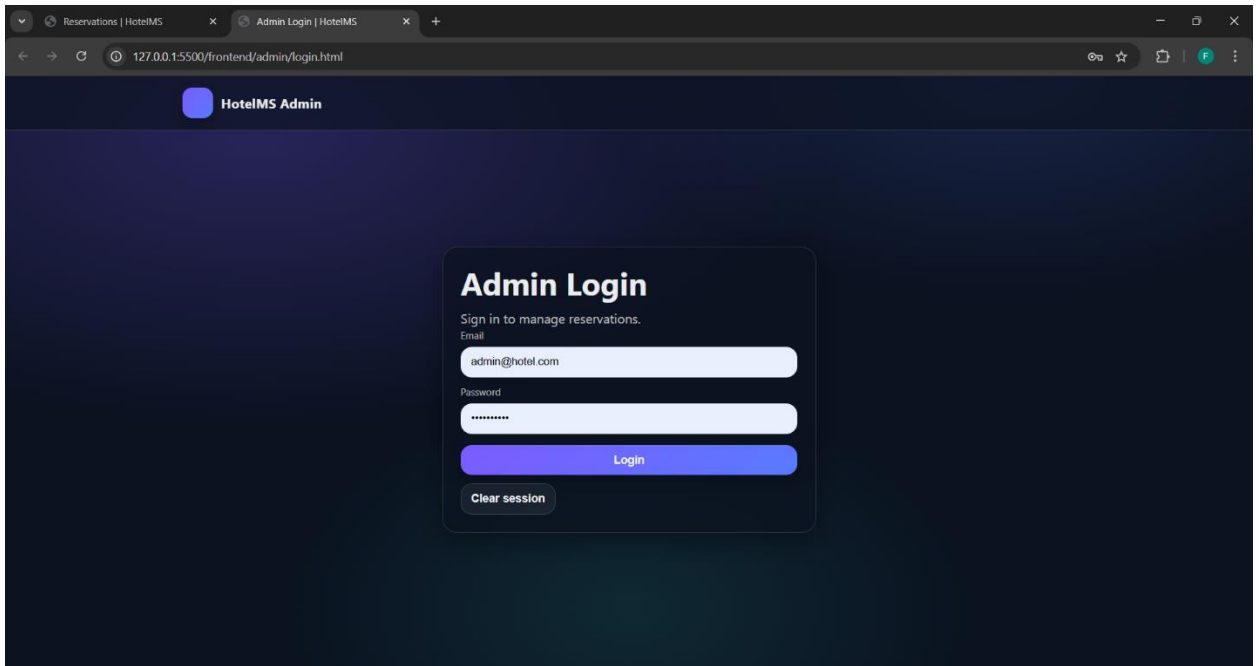


Figure 4: Admin Login

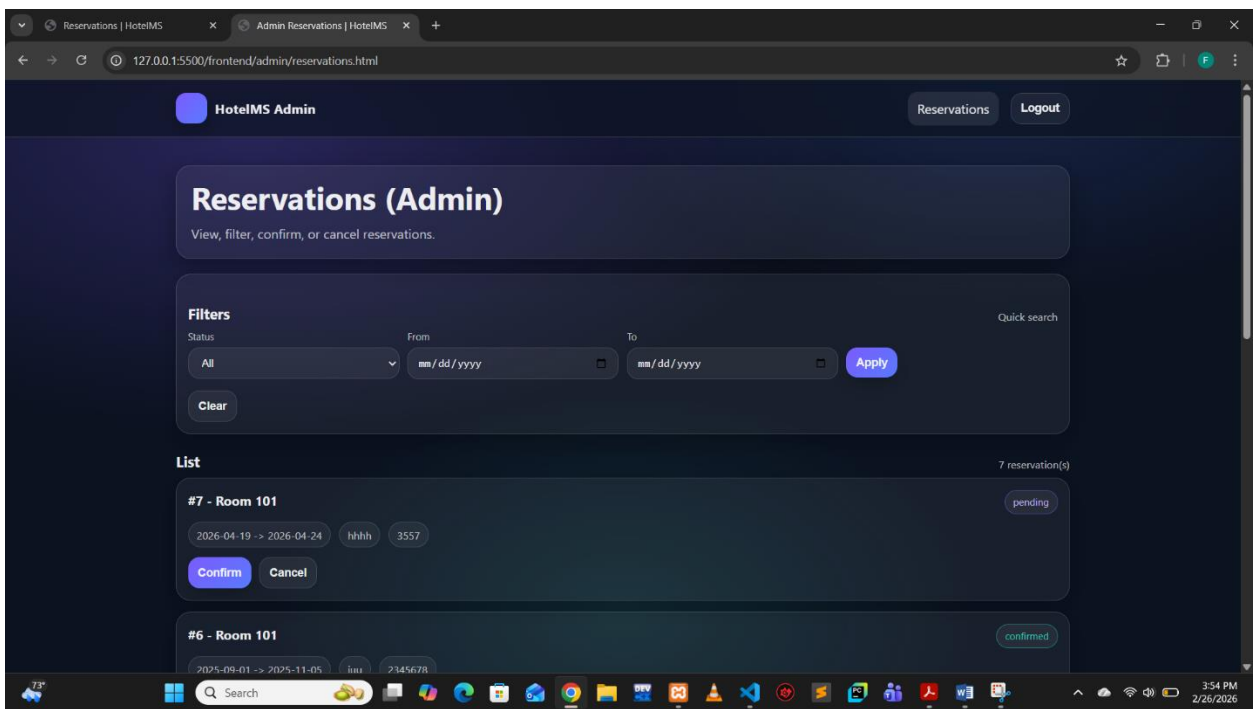


Figure 5: Admin Dashboard

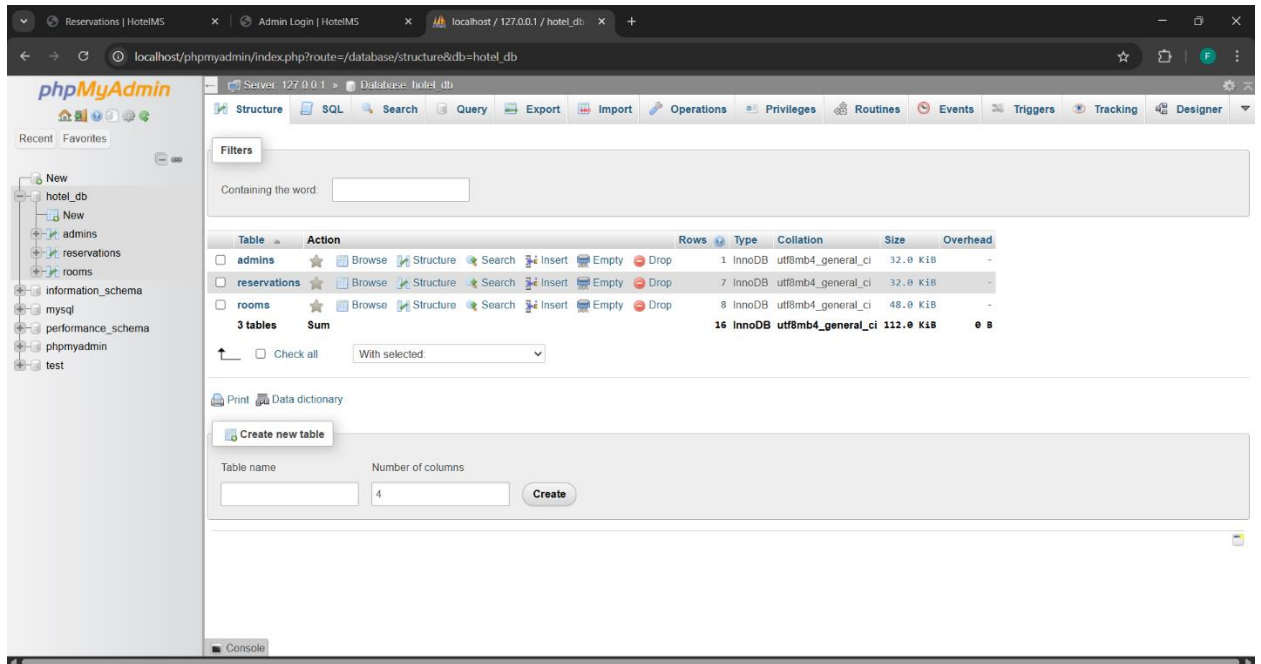


Figure 6: Database Tables

II. Explanation of major features working

Room Display

Rooms are successfully retrieved from the database and displayed with their details, including type, price, capacity, and status.

Reservation Creation

Customers can create reservations by filling out the booking form. The information is validated and stored correctly in the database.

Admin Login

Administrators can log in securely. Passwords are verified using hashing, and a token is generated for authenticated access.

Reservation Management:

Administrators can view, confirm, and cancel reservations. Status updates are saved in the database and reflected immediately in the dashboard.

III. Future Work

Improvements that can be done for the system to increase its efficiency are

- Online payment system
- Advanced reporting system
- Customer account management

Github- <https://github.com/Sitraton/HOTEL-MANAGEMENT-SYSTEM.git>

Video- <https://youtu.be/tEvvrnQd2m0>